Demonstrate your understanding of the Euclidean Algorithm by performing the sequence of integer divisions with remainder that is necessary to calculate gcd (a, b) using the Euclidean algorithm.

```
import sys #Had problems using sympy on local notebook so these
commands ensure it is installed properly.
!{sys.executable} -m pip install sympy

import sympy as sp

def gcd_m(a, b):
    a = abs(a)
    b = abs(b)
    if b == 0:
        return a
    if a == 0:
        return b
    else:
        q = a // b
        r = a - b * q
        print(f"{a} = {b} × {q} + {r}")
        return gcd_m(b, r)


gcd_m(2250,3533)
```

```
Requirement already satisfied: sympy in c:\users\rwgtv\anaconda3\lib\
site-packages (1.14.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\rwgtv\
anaconda3\lib\site-packages (from sympy) (1.3.0)
2250 = 3533 × 0 + 2250
3533 = 2250 × 1 + 1283
2250 = 1283 × 1 + 967
1283 = 967 × 1 + 316
967 = 316 × 3 + 19
316 = 19 × 16 + 12
19 = 12 × 1 + 7
12 = 7 × 1 + 5
7 = 5 × 1 + 2
5 = 2 × 2 + 1
2 = 1 × 2 + 0

1
```

Explanation The Euclidean Algorithm allows us to calculate the greatest common divisor of two integers. This is made possible by dividing the larger integer by the smaller integer and calculating the remainder. The algorithm then uses the smaller integer from the first calculation and the remainder to repeat until there is a remainder of zero, the final integer for b is the greatest common divisor of a and b.

Code Firstly, I define the function gcd_m which takes the arguments a and b which are both integers. def gcd_m(a,b): Next I use the abs function, which transforms negative integers into positive ones by calculating their distances from zero. Looking back at the formula for the Euclidean algorithm

$$gcd(a,b)=gcd(b,r)$$

when

$$a=qb+r.$$

we can see that the greatest common divisor of a is simply b when a is equal to zero. Because the greatest common divisor of zero and b is simply b. This applies if either a or b is zero. I calculate the values of q and r to complete the algorithm. For q I use the quotient divide built in Python using //. Here I use the formula from before to see that the quotient is equal to a divided by b. And r (the remainder) from this division is calculated by rearranging the formula. Finally, using recursion, I pass the results of these calculations back into the algorithm so it can be repeated until either value is equal to zero which then terminates the algorithm and returns the greatest common divisor of a and b.

Use a code cell and an appropriate command from the library Sympy to determine the prime factorizations of a and b.

```
def prime_factor(a, b):
    factors_of_a = sp.factorint(a)
    factors_of_b = sp.factorint(b)
    return factors_of_a, factors_of_b


prime_factor(2250, 3533)
```

```
({2: 1, 3: 2, 5: 3}, {3533: 1})
```

Every positive integer can be written as a product of primes, this is called prime factorisation. Each number has factors that are unique and there is only one way of representing a number using prime factors.

In the case with

$$a=2250\,b=3533$$

we can break these into their prime factors and get

$$a=2250=2^1\times3^2\times5^3$$

and

$$b=3533=3533^1$$

. Typically, by using the common prime factors of each number we can calculate the GCD by using the smallest power of each prime factor, multiplied together these prime factors will

result in the largest integer that divides both a and b. However, because 2250 an 3533 share no common prime factors, namely because b is a prime number, the GCD of the two values is 1.

Code To calculate the prime factors using the Sympy module, I created a function so it can be reused later if needed, which takes the arguments a and b. These are the numbers we wish to find the prime factors of.

To return the prime factors I used a function called factorint, which returns the primes that make up each integer uniquely and returned them when the function is called. This gives us a dictionary containing the prime factors of a and b.

Demonstrate your understanding of the Extended Euclidean Algorithm by determining the values of integer coefficients x and y that satisfy gcd(a, b) = xa + yb.

```
def mygcdex(a,b):
    if b==0:
        return (1,0,a)
    else:
        q = a // b
        r = a % b
        (x,y,d) = mygcdex(b,r)
        x1 = y
        y1 = x - q * y
        return (x1,y1,d)

mygcdex(2250,3533)
```

```
(-1487, 947, 1)
```

Explanation The Extended Euclidean Algorithm not only provides the Greatest Common Divisor of the two numbers but it also produces the integer coefficients x and y. This is so the formula is true.

$$d = a\,x + b\,y$$

where d is the greatest common divisor of a and b. The integer coefficents x and y show what numbers multiplied by a and b result in the GCD. The point is that every remainder can be made of a combination of a and b. This process repeats until the remainder is 0 and the algorithm terminates.

Code Firstly, the function is defined with the usual arguments, a and b. Then an initial check to see if b is zero, if b is zero then the algorithm should terminate and return a. This is because if b is zero the greatest common divisor of 0 and a is simply a, so the linear coefficients x would be 1 and y would be 0 as

$$a \cdot 1 + b \cdot 0 = a.$$

Next, we set the quotient, q, as usual by flooring the division of a over b using the // operator. The remainder is different however, because b could be positive or negative, pythons modulo operator handles the positive or negative result of the initial calculation so there is no need to worry about handling the remainder and making sure it is positive.

For the linear coefficients the recursive nature of the function means that for every step in the algorithm as the remainder decreases in size, it keeps track of how each remainder can be described using the original values of a and b. This continues until the gcd is found and the final values of the linear combination are found that satisfy the condition in the case of 2250, 3533 that.

$$1 = 2250 \cdot -1487 + 3533 \cdot 947$$

Use a text block to explain how to alter the definition of the Python function so that it produces a different pair of coefficients (x, y) from the pair produced in question NT.03, but that still satisfies the condition gcd(a, b) = $xa + yb$.

To allow the program to output different coefficient values for x and y, before returning the final values, an integer could be used to scale the coefficients using a formula to get different coefficients that still meet the requirements of the formula. Once we have the final values of x and y, in this case -1487 and 947, we can use these in combination with the gcd and a and b to produce new linear coefficients that satisfy the original formula.

$$x = x_1 - Z_n \cdot \frac{b}{d}$$

$$y = y_1 + Z_n \cdot \frac{a}{d}$$

We could either ask the user or randomly generate an integer at the start of the function, then before the coefficients are returned, modify them using the formula above to produce new and random coefficients, x and y that still satisfy the the conditions that gcd(a,b) = xa + yb.

```
def bezouts_coefficients(a,b):
    num_of_x_y = int(input("Enter number of coefficients"))
    for x_y in range(num_of_x_y):
        x_y_d = mygcdex(a,b)
        x2 = x_y_d[0] - x_y * b / x_y_d[2]
        y2 = x_y_d[1] + x_y * a / x_y_d[2]
        if x_y_d[2] == a * x2 + b * y2:
            print(x2, y2, " is a Valid Linear Combination")
        else:
            print("Error")
bezouts_coefficients(2250,3533)
```

```
Enter number of coefficients 100
```

```
-1487.0 947.0  is a Valid Linear Combination
-5020.0 3197.0  is a Valid Linear Combination
-8553.0 5447.0  is a Valid Linear Combination
-12086.0 7697.0  is a Valid Linear Combination
-15619.0 9947.0  is a Valid Linear Combination
-19152.0 12197.0  is a Valid Linear Combination
-22685.0 14447.0  is a Valid Linear Combination
-26218.0 16697.0  is a Valid Linear Combination
-29751.0 18947.0  is a Valid Linear Combination
```

-33284.0 21197.0 is a Valid Linear Combination
-36817.0 23447.0 is a Valid Linear Combination
-40350.0 25697.0 is a Valid Linear Combination
-43883.0 27947.0 is a Valid Linear Combination
-47416.0 30197.0 is a Valid Linear Combination
-50949.0 32447.0 is a Valid Linear Combination
-54482.0 34697.0 is a Valid Linear Combination
-58015.0 36947.0 is a Valid Linear Combination
-61548.0 39197.0 is a Valid Linear Combination
-65081.0 41447.0 is a Valid Linear Combination
-68614.0 43697.0 is a Valid Linear Combination
-72147.0 45947.0 is a Valid Linear Combination
-75680.0 48197.0 is a Valid Linear Combination
-79213.0 50447.0 is a Valid Linear Combination
-82746.0 52697.0 is a Valid Linear Combination
-86279.0 54947.0 is a Valid Linear Combination
-89812.0 57197.0 is a Valid Linear Combination
-93345.0 59447.0 is a Valid Linear Combination
-96878.0 61697.0 is a Valid Linear Combination
-100411.0 63947.0 is a Valid Linear Combination
-103944.0 66197.0 is a Valid Linear Combination
-107477.0 68447.0 is a Valid Linear Combination
-111010.0 70697.0 is a Valid Linear Combination
-114543.0 72947.0 is a Valid Linear Combination
-118076.0 75197.0 is a Valid Linear Combination
-121609.0 77447.0 is a Valid Linear Combination
-125142.0 79697.0 is a Valid Linear Combination
-128675.0 81947.0 is a Valid Linear Combination
-132208.0 84197.0 is a Valid Linear Combination
-135741.0 86447.0 is a Valid Linear Combination
-139274.0 88697.0 is a Valid Linear Combination
-142807.0 90947.0 is a Valid Linear Combination
-146340.0 93197.0 is a Valid Linear Combination
-149873.0 95447.0 is a Valid Linear Combination
-153406.0 97697.0 is a Valid Linear Combination
-156939.0 99947.0 is a Valid Linear Combination
-160472.0 102197.0 is a Valid Linear Combination
-164005.0 104447.0 is a Valid Linear Combination
-167538.0 106697.0 is a Valid Linear Combination
-171071.0 108947.0 is a Valid Linear Combination
-174604.0 111197.0 is a Valid Linear Combination
-178137.0 113447.0 is a Valid Linear Combination
-181670.0 115697.0 is a Valid Linear Combination
-185203.0 117947.0 is a Valid Linear Combination
-188736.0 120197.0 is a Valid Linear Combination
-192269.0 122447.0 is a Valid Linear Combination
-195802.0 124697.0 is a Valid Linear Combination
-199335.0 126947.0 is a Valid Linear Combination
-202868.0 129197.0 is a Valid Linear Combination

```
-206401.0 131447.0  is a Valid Linear Combination
-209934.0 133697.0  is a Valid Linear Combination
-213467.0 135947.0  is a Valid Linear Combination
-217000.0 138197.0  is a Valid Linear Combination
-220533.0 140447.0  is a Valid Linear Combination
-224066.0 142697.0  is a Valid Linear Combination
-227599.0 144947.0  is a Valid Linear Combination
-231132.0 147197.0  is a Valid Linear Combination
-234665.0 149447.0  is a Valid Linear Combination
-238198.0 151697.0  is a Valid Linear Combination
-241731.0 153947.0  is a Valid Linear Combination
-245264.0 156197.0  is a Valid Linear Combination
-248797.0 158447.0  is a Valid Linear Combination
-252330.0 160697.0  is a Valid Linear Combination
-255863.0 162947.0  is a Valid Linear Combination
-259396.0 165197.0  is a Valid Linear Combination
-262929.0 167447.0  is a Valid Linear Combination
-266462.0 169697.0  is a Valid Linear Combination
-269995.0 171947.0  is a Valid Linear Combination
-273528.0 174197.0  is a Valid Linear Combination
-277061.0 176447.0  is a Valid Linear Combination
-280594.0 178697.0  is a Valid Linear Combination
-284127.0 180947.0  is a Valid Linear Combination
-287660.0 183197.0  is a Valid Linear Combination
-291193.0 185447.0  is a Valid Linear Combination
-294726.0 187697.0  is a Valid Linear Combination
-298259.0 189947.0  is a Valid Linear Combination
-301792.0 192197.0  is a Valid Linear Combination
-305325.0 194447.0  is a Valid Linear Combination
-308858.0 196697.0  is a Valid Linear Combination
-312391.0 198947.0  is a Valid Linear Combination
-315924.0 201197.0  is a Valid Linear Combination
-319457.0 203447.0  is a Valid Linear Combination
-322990.0 205697.0  is a Valid Linear Combination
-326523.0 207947.0  is a Valid Linear Combination
-330056.0 210197.0  is a Valid Linear Combination
-333589.0 212447.0  is a Valid Linear Combination
-337122.0 214697.0  is a Valid Linear Combination
-340655.0 216947.0  is a Valid Linear Combination
-344188.0 219197.0  is a Valid Linear Combination
-347721.0 221447.0  is a Valid Linear Combination
-351254.0 223697.0  is a Valid Linear Combination
```

Consider the prime number p = 113. Use the result of Fermat's Little Theorem to calculate the value of 3 n (mod p).

Explanation Fermat's little theorem states that if p is prime and a is a positive integer not divisable by p then the following formula stands:

$$a^{p-1} \equiv 1 (mod\ p)$$

Where p = 113 and n = 22503533 and a = 3, we can begin by stating

$$a^{113-1} \equiv 1 \, mod \, 113 = ¿ a^{112} \equiv 1 \, mod \, 113$$

Now we need to reduce the exponent, Fermats's little theorum means that we do not need to work out the large exponent n we just need to know the remainder left by n when divided by p-1 or 112.

To start, we need to calculate 22503533 divided by 112 and its remainder, to do this we can work backwards slowly to calculate multiples of 112 and subtract from the remainder until we reach the final number of times 112 goes into 22503533 and what remainder is left over.

$$112 \cdot 200,000 = 22,400,000$$

```
112 * 200000
```

```
22400000
```

```
22503533 - 22400000
```

```
103533
```

We can then subtract this from our n value which is now our remainder, we can repeat the calculation to reduce the remainder again.

$$112 \cdot 900 = 100,800$$

$$103,533 - 100,800 = 2,733$$

Now we are getting closer to the remainder.

$$112 \cdot 20 = 2,240$$

$$2,733 - 2,240 = 493$$

$$112 \cdot 4 = 448$$

$$493 - 448 = 45$$

So we can see that

$$112 \cdot 200,924 + 45 = 22503533$$

```
print(112 * 200000, 22503533 - 22400000, 112 * 900, 103533 - 100800,
112 * 20, 2733 - 2240, 112 * 4, 493 - 448, 112 * 200924 + 45)
sp.Mod(22503533, 112)
```

```
22400000 103533 100800 2733 2240 493 448 45 22503533
```

```
45
```

$$22503533 \equiv 45 \, mod \, 112$$

Therefore, we get:

$$3^{22\,503533} \equiv 3^{45} (mod\, 113).$$

Now we know that 3n is congruent to 345 mod 113, we can use repeating squares to work out the value of 345 mod 113 but for this example we will use Sympy to simplify the process. Firstly, we will calculate 345 then we use this value to calculate what 322503533 is congruent to modulo 113.

```
3**45
```

```
295431270655083369864 3
```

```
sp.Mod(3**45, 113)
```

```
55
```

Our final result is:

$$3^{22503533} \equiv 55 (mod\, 113)$$

NT.06

Use the method of repeated squaring (also known as fast exponentiation) to calculate the value of 5 n (mod p), without any reference to Fermat's Little Theorem or Euler's Theorem.

In the previous question we made use of Fermat's little theorem to calculate the value of an mod p, this time we will use the process known as repeating squares to calculate the value of 5nmod p.

$$5^{22503533} (mod\, 113)$$

```
def repeated_squaring(base, exp, mod):
    result = 1
    base = base % mod
    while exp > 0:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result

repeated_squaring(5, 22503533, 113)
```

```
103
```

To implement repeated squaring, we begin by taking the base number, in our case 5, the exponent which is 22503533 and the modulo which is 113. We store the result starting at 1 because any number x^0 = 1.

Next we reduce the base modulo 113, to keep the numbers as small as possible. In our case 5 mod 113 = 5 anyway but for larger bases this is required.

$$5 (mod\, 113) = 5$$

Then while the exponent is a positive integer, we use a bitwise AND to check if the least significant bit of the exponent is equal to 1, if that is the case then the number will be an odd number.

We can see that the number 22503533, equals to this binary value which shows the powers of 2 required to reach this number, which is critical in repeated squaring.

$$22503533 = 1010101110110000001101101$$

If the least significant bit is 1 then we multiply the current result by the current base and reduce modulo 113, this includes the current exponent value into the base, multiplying it by that exponent of 2.

Regardless if the bit is 1 or 0, we then square the base and reduce again, each time to keep the number as small as possible, the repeated squaring occurs here and repeats until the while loop terminates, in cases where the bit is 0 the bit will be equal to 1 so nothing changes but the next bit is shifted to continue the operation. This excludes bits that are 0 from the exponent, allowing for the calculation of the next exponent.

Then we shift the exponent by 1 bit, which removes the least significant bit we just used to calculate the next base value. This process repeats until the exponent is 0, which terminates the exponent and allows for the return of the final result.

$1010101110110000001101101 = ¿\, 101010111011000000110110 = ¿\, 10101011101100000011011 = ¿\, 1010101110$

Finally, we return the result value that is calculated at each iteration of the while loop.

Determine the 64-bit binary string given by the ASCII encoding of your 8-digit Man Met university ID number (see the supporting notebook for further details). Assign this string to the variable m in your notebook.

```
m_ascii = "22503533"
m = []
for char in m_ascii:
    m.append((format(ord(char), '08b')))

m = ''.join(m)


k = "0011000100110010001100110011010000110101001101100011011100111000"
```

Firstly, I converted my student ID number, 22503533, stored in the m_ascii variable, into its binary representation using the ord function which returns the corresponding ASCII value for the character, then formatting this value into an 8 bit binary number using the format function. This output is then appended to the message list and then added to the string m using the join function. Thus we are left with a 64-bit represensation of the 8-bit student ID number.

Determine the result of applying the Initial Permutation to your message. Store the resulting 64-bit binary string in the variable IPm in your notebook.

```
INITIAL_PERMUTATION_TABLE = [
    58, 50, 42, 34, 26, 18, 10,  2,
    60, 52, 44, 36, 28, 20, 12,  4,
    62, 54, 46, 38, 30, 22, 14,  6,
    64, 56, 48, 40, 32, 24, 16,  8,
    57, 49, 41, 33, 25, 17,  9,  1,
    59, 51, 43, 35, 27, 19, 11,  3,
    61, 53, 45, 37, 29, 21, 13,  5,
    63, 55, 47, 39, 31, 23, 15,  7
]
IPm = []

for i in range(len(INITIAL_PERMUTATION_TABLE)):
    IPm.append(m[INITIAL_PERMUTATION_TABLE[i] - 1])

IPm = ''.join(IPm)
print(IPm)
```

```
0000000011111111001001001111010000000001111111100000000011010011
```

To perform the initial permutation that is required in the DES algorithm, firstly the table found in Stallings is converted to a list object and stored so we can iterate over it using a for loop. Then an empty list called IPm where we shall store the bit permutation is declared. Next we look through the length of the ip_table, which is 64 entries so 64-bits, which is required as the input length for DES. For each entry in the list we append the value stored in the position of the ip_table i - 1 (to account for the start at 0 count in Python), to the position found in the original binary message. Now once we have the completed list, we can utilise the join function to transform the IPm form a list into the proper string type required for the next step in the DES algorithm.

Determine the left and right halves that are the inputs to the first round. Store these 32-bit binary strings in the variables L0 and R0 respectively.

```
L0 = IPm[:32]
R0 = IPm[32:]

print(L0,R0)
```

```
00000000111111110010010011110100 00000000111111110000000011010011
```

This code splits the 64 bit string into the left and right 32-bit strings that will be used in round one of the DES algorithm.

Determine the Expansion/permutation of and store the resulting 48-bit binary string in the variable ER0.

```
EXPANSION_TABLE = [
    32,    1,   2,   3,   4,   5,
    4,     5,   6,   7,   8,   9,
    8,     9,   10,  11,  12,  13,
    12,    13,  14,  15,  16,  17,
```

```
    16, 17,      18,   19,   20,   21,
    20,    21,   22,   23,   24,   25,
    24,    25,   26,   27,   28,   29,
    28,    29,   30,   31,   32,   1
]
ER0 = []

for i in range(len(EXPANSION_TABLE)):
    ER0.append(R0[EXPANSION_TABLE[i] - 1])

ER0 = ''.join(ER0)
print(ER0)
print(len(ER0))
```

```
100000000001011111111110100000000001011010100110
48
```

The expansion table allows the bits within the R0 32-bit string to be mapped across, sometimes twice to a new 48-bit string. The code iterates through the length of the expansion table and appends the value at each point in R0 to the new variable ER0.

DES.05 Determine the first subkey and store the 48-bit binary string in the variable K1.

```
PC1 = [
    57, 49, 41, 33, 25, 17, 9,
    1,  58, 50, 42, 34, 26, 18,
    10, 2,  59, 51, 43, 35, 27,
    19, 11, 3,  60, 52, 44, 36,
    63, 55, 47, 39, 31, 23, 15,
    7,  62, 54, 46, 38, 30, 22,
    14, 6,  61, 53, 45, 37, 29,
    21, 13, 5,  28, 20, 12, 4
]

pc1_key_str = ''.join(k[bit - 1] for bit in PC1)

C0 = pc1_key_str[:28]
D0 = pc1_key_str[28:]


C1 = C0[1:] + C0[:1]
D1 = D0[1:] + D0[:1]

PC2 = [
    14, 17, 11, 24, 1, 5,
    3, 28, 15, 6, 21, 10,
    23, 19, 12, 4, 26, 8,
    16, 7, 27, 20, 13, 2,
    41, 52, 31, 37, 47, 55,
```

```
    30, 40, 51, 45, 33, 48,
    44, 49, 39, 56, 34, 53,
    46, 42, 50, 36, 29, 32
]

CD1 = C1 + D1


K1 = ''.join(CD1[item - 1] for item in PC2)
```

To create the first subkey, K1, first we apply the first permuted choice (PC-1) which will transform the original key, K, by reordering and reducing the 64-bit key into a 56-bit key, removing every 8th parity bit. Then we split this 56-bit key into two 28-bit left and right halves, C0 and D0. We then use a circular left shift by 1 bit on both the key halves. The two key halves are then added back together into the variable CD1, After this the second permuted choice (PC-2) is applied to CD1 to create the first subkey, K1.

Determine the XOR operation on the inputs ER0 and K1. Store the resulting 48-bit binary string the variable A.

```
def XOR(a,b):
    A = ''.join(str((int(x) + int(y))%2) for x,y in zip(a,b))
    return A

A = XOR(K1, ER0)
```

To perform XOR operations, which happen a couple of times during DES, an XOR function is defined which takes two binary numbers a,b then transforms them into integers then into strings after they're divided modulo 2, giving 0 or 1. These values are then joined together to produce an output variable A.

Determine the result of applying the S-box substitution/choice operation to your bit string . Store the resulting 32-bit binary string in the variable SA.

```
s_boxes = [
    [
        [14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7],
        [0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8],
        [4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0],
        [15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13]
    ],
    [
        [15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10],
        [3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5],
        [0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15],
        [13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9]
    ],
    [
        [10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8],
```

```
        [13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1],
        [13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7],
        [1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12]
    ],
    [
        [7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15],
        [13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9],
        [10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4],
        [3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14]
    ],
    [
        [2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9],
        [14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6],
        [4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14],
        [11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3]
    ],
    [
        [12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11],
        [10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8],
        [9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6],
        [4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13]
    ],
    [
        [4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1],
        [13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6],
        [1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2],
        [6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12]
    ],
    [
        [13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7],
        [1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2],
        [7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8],
        [2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11]
    ]
]


bit_blocks = []

for i in range(0, len(A), 6):
    bit_blocks.append(A[i:i+6])

SA = ''

for i in range(8):
    block = bit_blocks[i]

    row = int(block[0] + block[5], 2)
    col = int(block[1:5], 2)
```

```
    s_value = s_boxes[i][row][col]

    s_bits = format(s_value, '04b')

    SA += s_bits

print(SA)
print(len(SA))
```

```
10011101100000100000111010010100
32
```

The S-Box substituition boxes are defined which will be used later to perform the substituitions. First an empty list is created which will be used to store the 6-bit binary blocks which the S-box substituition will be performed on. To create the 6-bit blocks we can loop over the length of A and every 6th item we can append the values in A up to that point to get 8 equally sized 6-bit blocks from the original A binary string. Then we define an empty string where we will store our final value of SA. Now a loop is used to go through each block and take the first and last item to make the row and the middle four bits to create the column, these values are then selected from the s-boxes and stored in the s_value variable. We then format these bits to leave us with 4-bit strings which are then appended to the SA string variable, until a 32-bit string is created.

Determine the result of applying the permutation function to SA and store the resulting 32-bit binary string the variable PSA.

```
P = [
    16, 7, 20, 21,
    29, 12, 28, 17,
    1, 15, 23, 26,
    5, 18, 31, 10,
    2, 8, 24, 14,
    32, 27, 3, 9,
    19, 13, 30, 6,
    22, 11, 4, 25
]

PSA = ''

for pos in P:
    PSA += SA[pos - 1]

print(PSA)
```

```
00010010111010000100000100111011
```

The final permutation function is applied to SA by again iterating through each position within the P table and selecting the bits, reducing the index by 1 each time.

Determine the two 32-bit binary left- and right-hand halves of the output strings for round one. Store these in the variables L1 and R1 respectively. Compare your final output with the final output strings given on Moodle to confirm the correctness of your work.

```
L1 = R0
R1 = XOR(L0, PSA)
print(L1, R1)
print(L1 + R1)
check_string =
'00000000111111110000000011010011000100100001011101100101110011111'
if (L1 + R1 == check_string):
    print("True")
```

Finally, to get our first round output of L1 and R1 we set L1 to equal the value for R0 as seen in the Fiestal network for DES. Then we set the R1 to equal the XOR output of L0 and the PSA variable, again replicating the notation seen in the Feistal network diagrams for DES. This produces our final output, which we can stitch together into an unbroken string to check against the check string provided on Moodle.

$$n=22503533\,,a=2250\,,b=3533$$

$$A=2250\,(mod\,256)=202$$

$$B=3533\,(mod\,256)=205$$

Therefore the corresponding polynomials for A and B and M are as follows:

$$P_A(x)=x^7+x^6+x^3+x$$

$$P_B(x)=x^7+x^6+x^3+x^2+1$$

$$m(x)=x^8+x^4+x^3+x+1$$

Carry out the sequence of polynomial divisions with remainder of the Euclidean algorithm for polynomials, to confirm that gcd($PA\,(x)$, $PB\,(x)$) is indeed 1.

To start with we are going to operate on the principle of m(x) / PA(x). To calculate the greatest common divisor we use the Euclidean Algorithm which states that:

$$P_A(x)=m(x)\cdot q(x)+r(x)$$

$$\left(x^8+x^4+x^3+x+1\right)\div\left(x^7+x^6+x^3+x\right)$$

Start by dividing the larger value of m(x) by PA(x) to find the quotient and the remainder. We do this by taking the leading term of m(x) and subtracting the leading term of PA(x) to get the first part of the quotient which we use to multiply by the remainder and subtract from m(x) for the division step 1.

$$x^8-x^7=x\cdot\left(x^7+x^6+x^3+x\right)=x^8+x^7+x^4+x^2¿$$

$$x^8+x^4+x^3+x+1-x^8+x^7+x^4+x^2¿=x^7+x^3+x^2+x+1$$

Then we continue by subtracting the leading terms again and repeating the process, we do this until the degree of the divisor is less than the leading degree term of the divisor. This terminates the division and gives us our first quotient and first remainder.

$$x^7 - x^7 = 1 \cdot \left(x^7 + x^6 + x^3 + x\right) = x^7 + x^6 + x^3 + x$$

$$x^7 + x^3 + x^2 + x + 1 - x^7 + x^6 + x^3 + x = x^6 + x^2 + 1$$

The degree x^6 is less than the divisors leading term x^7 so the division results in:

$$Q(x) = x + 1$$

$$r(x) = x^6 + x^2 + 1$$

We now continue with the algorithm using the old remainder as the new value and the new remainder as the divisor. Again we start with the leading terms and continue from there to get the quotient and the divisor.

$$\left(x^7 + x^6 + x^3 + x\right) \div \left(x^6 + x^2 + 1\right)$$

$$x^7 - x^6 = x \cdot \left(x^6 + x^2 + 1\right) = x^7 + x^3 + x$$

$$x^7 + x^6 + x^3 + x - x^7 + x^3 + x = x^6$$

Quotient remains the same start point with just x. Continue again with the new leading terms.

$$x^6 - x^6 = 1 \cdot \left(x^6 + x^2 + 1\right) = x^6 + x^2 + 1$$

$$x^6 - x^6 + x^2 + 1 = x^2 + 1$$

Again we have the quotient of x+1 with the remainder of x^2 + 1 which is smaller than the current divisors first term so we finish this division and repeat the algorithm by replacing the old polynomial values with the new ones to satisfy the algorithm.

$$x^6 + x^2 + 1 \div x^2 + 1$$

Take the leading terms as usual.

$$x^6 - x^2 = x^4 \cdot \left(x^2 + 1\right) = x^6 + x^4$$

$$x^6 + x^2 + 1 - x^6 + x^4 = x^4 + x^2 + 1$$

And repeat again.

$$x^4 - x^2 = x^2 \cdot \left(x^2 + 1\right) = x^4 + x^2$$

$$x^4 + x^2 + 1 - x^4 + x^2 = 1$$

Therefore the quotient is x + 1 and the remainder is 1.

Determine the product polynomial $PA\,(x) \cdot PB\,(x)$, where the product is carried out with the multiplication operation of GF(2^8).

With the polynomials of A and B identified then next step in AES is to multiply the polynomials then reduce the result by the irreducble polynomial if the result of the polynomial multiplication has exponents with degrees larger than 7. Like with any field, the result must "wrap" back around so the value fits within 8 bits and as such is within the GF(2^8). The following is the irreducible polynomial which we use to reduce the polynomial modulo:

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

So to begin, we perform normal polynomial multiplication but the operations on the coefficients are modulo 2, because the coefficients in 2^8 can only be in the set {0,1}.

$$P_A(x) \cdot P_B(x) = \left(x^7 + x^6 + x^3 + x\right) \cdot \left(x^7 + x^6 + x^3 + x^2 + 1\right)$$

Now, we have to multiply the polynomials together and reduce by modulo 2. I used a table to solve this polynomial multiplication as follows. | |

x^7

|

x^6

|

x^3

|

x

| |----------|----------|----------|----------|---------|| x^7 |

x^{14}

|

x^{13}

|

x^{10}

|

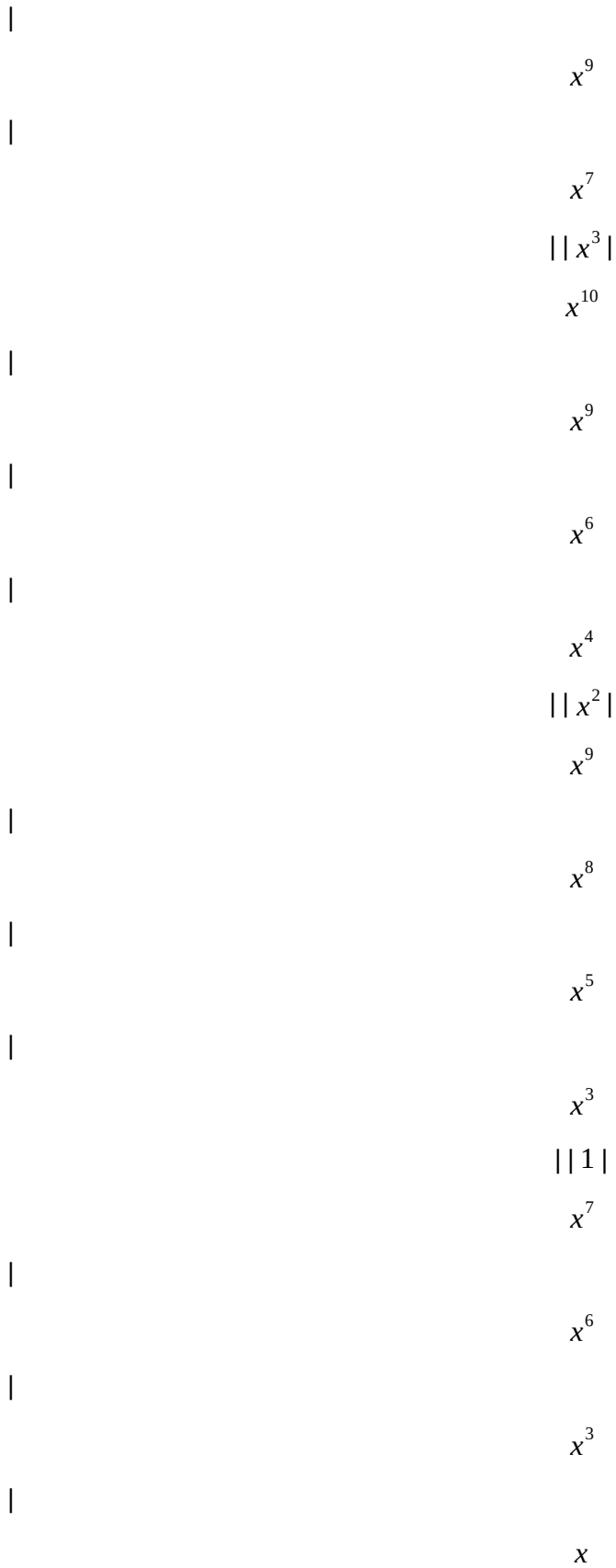
x^8

|| x^6 |

x^{13}

|

x^{12}


x^9
x^7
x^3
x^{10}
x^9
x^6
x^4
x^2
x^9
x^8
x^5
x^3
1
x^7
x^6
x^3
x

|

The table shows the multiplication of each polynomial, now we must work out the final output polynomial to see if it needs to be reduced by the irreducible polynomial. So we collect the terms, because we are using modulo 2, the coefficients must be either 0 or 1. So if there are 2 terms then they cancel eachother out because 2 mod 2 is 0.

$$1x^{14}+2x^{13}+1x^{12}+2x^{10}+3x^{9}+2x^{8}+2x^{7}+2x^{6}+1x^{5}+1x^{4}+2x^{3}+x$$

Now again because we are using modulo 2, 1 becomes 1, 2 becomes 0 and 3 becomes 1. So the terms then become:

$$x^{14}+x^{12}+x^{9}+x^{5}+x^{4}+x$$

As we can see the polynomial contains exponents that are larger than or equal to 8, so we must reduce the polynomial by the irriducible polynomial m(x). To do this, we can take instances of x^8 and replace them with the following:

$$x^{8}\equiv\left(x^{4}+x^{3}+x+1\right)$$

So to take the largest exponent first and work down until we have the final output.

$$x^{8}\cdot x^{6}=x^{14}=¿\,x^{6}\left(x^{4}+x^{3}+x+1\right)=¿\left(x^{10}+x^{9}+x^{7}+x^{6}\right)$$

Now we put this back into the polynomial and cancel out any terms.

$$\left(x^{10}+x^{9}+x^{7}+x^{6}\right)+x^{12}+x^{9}+x^{5}+x^{4}+x$$

Here the common terms are x^9 so we can remove them and repeat the process on the next term larger than or equal to 8, which is 12.

$$x^{12}+x^{10}+x^{7}+x^{6}+x^{5}+x^{4}+x$$

$$x^{8}\cdot x^{4}=x^{12}=¿\,x^{4}\left(x^{4}+x^{3}+x+1\right)=¿\left(x^{8}+x^{7}+x^{5}+x^{4}\right)$$

Again we put this back into the polynomial and cancel out the terms.

$$\left(x^{8}+x^{7}+x^{5}+x^{4}\right)+x^{10}+x^{7}+x^{6}+x^{5}+x^{4}+x$$

Becomes

$$x^{10}+x^{8}+x^{6}+x$$

$$x^{8}\cdot x^{2}=x^{10}=¿\,x^{2}\left(x^{4}+x^{3}+x+1\right)=¿\left(x^{6}+x^{5}+x^{3}+x^{2}\right)$$

Back into the polynomial and assess the terms.

$$\left(x^{6}+x^{5}+x^{3}+x^{2}\right)+x^{8}+x^{6}+x$$

Becomes:

$$x^{8}+x^{5}+x^{3}+x^{2}+x$$

$$x^8=\left(x^4+x^3+x+1\right)$$

Remove the like terms from:

$$\left(x^4+x^3+x+1\right)+x^5+x^3+x^2+x$$

Finally we get:

$$x^5+x^4+x^2+1$$

Determine the polynomial $Q(x)$ = (PA (x)) −1 , i.e. $Q(x)$ is the multiplicative inverse of PA (x) in GF(2^8). The polynomial $Q(x)$ can be found from the extended Euclidean algorithm.

To determine if Q(x) = (PA(x))^-1 is true we must perform the extended Euclidean algorithm. So basically, we want to know if Q(x) times Pa(x) is congruent to 1 modulo m(x). We use the following formula to check:

$$r_i=s_i\cdot m(x)+t_i\cdot P_A(x)$$

We can begin by stating that:

$$r_0=m(x)=x^8+x^4+x^3+x+1$$

$$r_1=P_A(x)=x^7+x^6+x^3+x$$

And now we can also state that:

$$s_0=1, s_1=0$$

$$t_0=0, t_1=1$$

$$1\cdot m(x)+0\cdot P_A(x)=r0$$

$$0\cdot m(x)+1\cdot P_A(x)=r1$$

We have our first Bezout coefficients to work from, now we can begin the algorithm by dividing r0 by r1, using the same technique of long division we did in the last question. So to start the leading terms are:

$$r_0=x^8, r_1=x^7$$

So we divide these values to get x, then multiply x by r1. We also add x as the first term in the quotient.

$$x\cdot r_1=x^8+x^7+x^4+x^2$$

Now we take this value away from r0 using XOR, basically mod 2 to get the remainder.

$$x^8+x^4+x^3+x+1-x^8+x^7+x^4+x^2=x^7+x^3+x^2+x+1$$

We can continue by taking the new leading term which is x^7 away from r1 which is also x^7 so we get 1, again we multiply this term by r1 and put the +1 into our quotient value.

$$1 \cdot r_1 = x^7 + x^6 + x^3 + x$$

$$x^7 + x^3 + x^2 + x + 1 - x^7 + x^6 + x^3 + x = x^6 + x^2 + 1$$

Now the remainder has a degree less than r1 so we stop the division here and we are left with the quotient of x+1 and the remainder of x^6+x^2+1. Now we have a new Bezout coefficient.

$$r_2 = s_2 \cdot m(x) + t_2 \cdot P_A(x)$$

$$x^6 + x^2 + 1 = 1 \cdot m(x) + (x+1) \cdot P\,A(x)$$

This completes step one of the algorithm and gives us our new values for r_1 and r_2. Next we continue and divide r_1 by r_2 the same way as before.

$$r_1 = x^7 + x^6 + x^3 + x, r_2 = x^6 + x^2 + 1$$

The leading term for r1 is x^7 and the leading term for r2 is x^6 which leaves our first quotient as x and when XOR'd leaves us with:

$$x \cdot r_2 = x^7 + x^3 + x$$

$$x^6$$

Now the leading term is x^6 and r_2 starts with x^6 so we get 1, leaving us with a quotient of x+1 and a new remainder of:

$$x^6 + x^2 + 1 - x^6 = x^2 + 1$$

Which we can use the second quotient to update the coefficients to become

$$s_3 = x + 1, t_3 = x + 1 \cdot x + 1 = x^2$$

$$x^2 + 1 = x + 1 \cdot m(x) + x^2 \cdot P_A(x)$$

We still haven't got to a remainder of 1 yet so we keep going until we do, this time using r_3 and r_2 as our values.

$$r_2 = x^6 + x^2 + 1, r_3 = x^2 + 1$$

The leading term is x^6 and x^2 so we are left with x^4, this multiplied by r_3 gives us:

$$x^4 \cdot (x^2 + 1) = x^6 + x^4$$

$$x^6 + x^2 + 1 - x^6 + x^4 = x^4 + x^2 + 1$$

We get q3 which has a new term x^4 and we continue with the division with the new term of x^4 as the leading term.

$$x^4 - x^2 = x^2 \cdot (x^2 + 1) = x^4 + x^2$$

$$x^4 + x^2 + 1 - x^4 + x^2 = 1$$

This means our quotient becomes x^4 + x^2 and our remainder is 1. This completes the division. So we can update the coefficients to get our final values which will be:

$$t_4 = t_2 \oplus q_3 \cdot t_3$$

$$s_4 = s_3 \oplus (x^4 + x^2)(x+1) = x^5 + x^4 + x^3 + x^2 + 1$$

$$t_4 = t_3 \oplus (x^4 + x^2) \cdot x^2 = (x+1) \oplus (x^6 + x^4) = x^6 + x^4 + x + 1$$

$$1 = x^5 + x^4 + x^3 + x^2 + 1 \cdot m(x) + x^6 + x^4 + x + 1 \cdot P_A(x)$$

$$Q(x) = (P_A(x))^{-1} \equiv x^6 + x^4 + x + 1 \, (mod\, m(x))$$

$$n = 22503533$$

$$a = 2250 \, b = 3533$$

1 Let p be the smallest prime satisfying $p \geq n$ and let q be the smallest prime satisfying $q \geq (n +$ 10^6). Let m denote the product $m = p \cdot q$. Determine the values p, q, m with the help of Sympy's nextprime function.

```
import sympy as sp
n = 22503533
a = 2250
b = 3533

q = sp.nextprime(n + 10 ** 6)

p = sp.nextprime(n)

m = p * q
print(q, p, m)
```

```
23503541 22503539 528912851531599
```

To get our primes, p and q we use the sympy modules built in nextprime function, which returns the nextprime in the sequence of primes after the integer provided. To get p we ask for the next prime that is larger than or equal to n and for q we ask for the next prime that is equal to n + 10 ^6. Then to get our m value, sometimes called n in Stallings and other books, we multiply our primes together.

Your public key is the pair (101, m). Determine your corresponding private key pair (d, m). Store the relevant value in the variable d.

```
#e = 101, phi_m = 528912805524520
phi_m = (p-1)*(q-1)
e = 101
```

```
(s, t, gcd) = sp.gcdex(101, phi_m)
d = s % phi_m

print(d, (d * e) % phi_m)
```

```
382283512903861 1
```

Now we know our public key pair because it is provided to us in the assessment specification we have our e value as 101 and our m is already defined from the last question. To get the phi(m) also known as Eulers totient, we calculate p-1 multiplied by q-1. Now with this value calculated we can use the Extended Euclidean Algorithm to check that our e is a valid value, that it is coprime to phi(m), if e was not coprime then it would not satisfy the requirements for RSA encryption and we would have to pick another value. Now from the Extended Euclidean Algorithm we can determine our d value, such that:

$$gcd(101, phi(m)) = 1$$

Use your public key to encrypt the message represented by your number b. Store the encrypted value in the variable M.

To get our encrypted plaintext, we must square the value of our plaintext which is b by the value selected before as e. This e value has to be coprime to the Eulers Totient or the encryption will not work. To increase the speed of this calculation and the computation requirements we can use the repeated squaring method by providing the modulo, in this case m, which we will reduce by to get our cipher text.

```
M = repeated_squaring(b, e, m)
print(M)
```

```
103
```

Finally, decrypt the encrypted message M with the use of your private key and confirm that the plaintext message b is recovered.

Because of the size of the exponents we are dealing with when decrypting in RSA, we need to make use of the Chinese Remainder Theorum to reduce the exponents to reduce the computational requirements of handling these large numbers, then we can use the same repeated squaring algorithm we have used previously to reduce the modular exponents further. The large private key is what gives RSA its security so the value has to remain large.

So we start with the following formulas:

$$d = k \cdot (p-1) + r \; r = d \, (mod \, p-1)$$

So for p-1:

$$p - 1 = 22503538$$

$$k_p = \frac{d}{p-1} = 16987707$$

$$d_p = d \pmod{p-1} = 2896495$$

$$k_p \cdot (p-1) + d_p = 16987707 + 2896495 = d$$

And again we repeat for q-1:

$$q - 1 = 23503540$$

$$k_q = \frac{d}{q-1} = 16264933$$

$$d_q = d \pmod{q-1} = 9541041$$

$$k_q \cdot (q-1) + d_q = 16253933 + 23503540 + 9541041 = d$$

So now we have the values for dp and dq.

$$d_p = 2896495, d_q = 9541041$$

Now we can calculate the smaller modular exponents which will be congruent to our larger exponents.

$$b1 = 239337581638586^{2896495} \pmod{22503539}, b2 = 239337581638586^{9541041} \pmod{23503541}$$

Now with these smaller modular exponents we can use the square and multiply algorithm to reduce the modular exponentiation. After we have computed the values for b1 and b2, we can see that they both equal to the original plaintext value of 3533. But if this was not the case we would have to complete the Chinese Remainder Theorum.

```
d_p = d % (p - 1)
d_q = d % (q - 1)

b1 = repeated_squaring(M, d_p, p)
b2 = repeated_squaring(M, d_q, q)

print(b1, b2)
```

```
3533 3533
```

Let p be defined as in Q1, i.e. the smallest prime satisfying $p \geq n$. Determine the smallest positive integer α such that α is a primitive root modulo p. Store this in a variable alpha. Your code should not use the Sympy function sympy.primitive_root() to find the smallest primitive root. Rather you should loop through the candidates and use the function sympy.is_primitive_root() to test each candidate to find the smallest root. You can then use sympy.primitive_root() to confirm the correctness of your answer.

```
n = 22503533

p = sp.nextprime(n)

for i in range(2,p):
    if sp.is_primitive_root(i, p):
        alpha = i
        break

print(alpha)
sp.primitive_root(p)
```

```
2
```

```
2
```

The primative root or primative element comes from a group of elements which contains all integers up to p-1. In group theory the set of elements Zn* has to contain integers which are inverses, if n is a prime number then every number that is not zero modulo p is co-prime to the prime number, p. This means that the set Zp* contains all integers except from 0, and p. This means that there is no zero, so we would assume the smallest primative root would be 1, but it cannot be 1 as 1 is the neutral element of the set, so we start from 2. In this code we check from 2 all the way to p to see if the value is a primative root, if it is then we set the variable alpha to equal that value and end the for loop using a break.

Alice and Bob agree to a Diffie-Hellman key exchange using the prime p and primitive root α. Alice's private key is $XA = a$ and Bob's private key is $XB = b$. Determine the public keys YA and YB exchanged by Alice and Bob. Store these in variables YA and YB.

```
from random import randint
a = randint(2, p-2)
b = randint(2, p-2)

YA = repeated_squaring(alpha, a, p)
YB = repeated_squaring(alpha, b, p)

print(YA, YB)
```

```
---------------------------------------------------------------------------
----
NameError                                 Traceback (most recent call
last)
Cell In[3], line 2
      1 from random import randint
----> 2 a = randint(2, p-2)
      3 b = randint(2, p-2)
      5 YA = repeated_squaring(alpha, a, p)

NameError: name 'p' is not defined
```

To implement the Diffie-Hellman Key Exchange we must first define the private values for both Alice and Bob, to do this I have used a random integer generater that will generate a random integer between 2 and p - 2, because the generator alpha is from a multiplicative group p-1. Then using these values we can create the public keys YA and YB which are exchanged in the public domain. To do this we compute:

$$YA = \alpha^a (mod\ p)$$

$$YB = \alpha^b (mod\ p)$$

Determine the shared secret key K that Alice and Bob can compute. Store this in a variable K. For each of the parts PKC.05 to 07 include a text cell with a brief explanation of how your code implements the Diffie-Hellman key exchange protocol specification

```
KprA = repeated_squaring(YB, a, p)
KprB = repeated_squaring(YA, b, p)
assert KprA == KprB
K = KprA
print(K)
```

Now the cleverness of the DHKE is that the values calculated for the private key K are the same due to the multiplication of the exponents being commutative.

$$B^a = \left(\alpha^b\right)^a = \alpha^{ab} (mod\ p)$$

$$A^b = \left(\alpha^a\right)^b = \alpha^{ab} (mod\ p)$$

So despite having different values for a and b, and neither of which are within the public domain the private key is able to be calculated on both sides because the secret key is the same when reduced modulo p on either side. The code achieves this by calculating the public key with the private value of both Alice and Bob modulo p, then asserting (checking) whether they are equal and if they are then set K to be the shared secret. And so the secret is created and both parties have it even after sharing details across an insecure channel of communication.

Let p be defined as in Q1, i.e. the smallest prime satisfying $p \geq n$. Confirm that p passes the Miller-Rabin test by carrying out the necessary computations to confirm the expected value(s) modulo p of the exponential(s) with the base a. Include a text cell with an explanation of how your code implements the Miller-Rabin technique outlined in the module.

```
n = 22503533
p = sp.nextprime(n)
a = randint(2, p - 2)

fermat_x = repeated_squaring(a, p - 1, p)

if fermat_x != 1:
    print("Number is composite by Fermat test")
else:
    print("Passed Fermat test")
```

```
exp = p - 1
r = 0

while exp % 2 == 0:
    exp = exp // 2
    r = r + 1

x = repeated_squaring(a, exp, p)

if x == 1 or x == p - 1:
    print("First Miller–Rabin condition satisfied")
else:
    for i in range(r - 1):
        x = repeated_squaring(x, 2, p)
        print(x)
        if x == p - 1:
            print("Probably prime (Miller–Rabin)")
            break
    else:
        print("Composite by Miller–Rabin")
```

```
Passed Fermat test
First Miller–Rabin condition satisfied
```

Before the Miller-Rabin test, Fermat's Theorum was used to test if a number was a prime or composite number.

$$a^{n-1} \equiv 1 (mod\, n)$$

However, there are numbers which are not primes which still satisfy this test when the value of a is known as a "liar" and the congruence still satisfies 1 modulo n, these are called Carmichael numbers. To fix this, the Miller-Rabin test looks at the structure of the exponent, instead of doing a^n-1 like Fermat, Miller-Rabin writes n - 1 in the form of 2^rd, where d is an odd number, it then computes a^d modulo n. From this value the test doubles the exponent to a^2r-1d, if any of these values are congruent to n - 1 or the first value is congruent to 1 then a is known as a witness to n being probably a prime number. If non of the values are congruent to 1 or n - 1 then the number n is a composite number.

We start with our usual student ID number, n and create our prime from the first prime after the number n. In this case, we know that p will be prime but we can use the Miller-Rabin test to see what values of a act as "witnesses" to p being prime. So we create a random integer a which we will check. We start by doing Fermat's Theorum, if the value is prime then it should be congruent to 1 mod p. Now we factor p - 1 as 2^rd, with d the smallest odd exponent and r the number of times p - 1 is divisible by 2.

We can then calculate the starting value a^d modulo p, if this is 1 or p - 1 then a is already a witness for the primality of p. If it is not then we continue and double the exponent to see if it ever does. If it doesn't then p is a composite number.